

Bus Booking System

OOP Design Review — C# ASP.NET Core

Dharshini K

May 2026 | Presidio × Genspark Training

Project: Bus Booking System (BBS) — Full-stack application built with ASP.NET Core 8 (backend) and Angular 18 (frontend). The backend follows a layered architecture: Models → Interfaces → Services → Controllers. This document analyses how OOP principles were applied, what was done well, and what could be improved.

1. Abstract Class

An **abstract class** defines a contract with optional shared implementation. Concrete subclasses must fill in the blanks while inheriting ready-made members.

1.1 BaseEntity — The Domain Root

Every EF Core entity in the project inherits from **BaseEntity**. This single class removes the need to redeclare *Id* and *CreatedAt* in all 8+ model files.

```
// Models/BaseEntity.cs

public abstract class BaseEntity
{
    public int Id { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

// All domain models inherit it:

public class User : BaseEntity { ... } // Id + CreatedAt free

public class Bus : BaseEntity { ... } // Id + CreatedAt free

public class BusOperator : BaseEntity { ... } // Id + CreatedAt free

public class Booking : BaseEntity { ... } // Id + CreatedAt free

public class BusSchedule : BaseEntity { ... } // Id + CreatedAt free
```

- ✓ **DRY — eliminated 16+ redundant property declarations across model files.**
- ✓ **'abstract' keyword prevents accidental direct instantiation of BaseEntity.**
- ✓ **EF Core respects the abstract modifier — no 'BaseEntities' table is created.**

1.2 AppControllerBase — The Shared Controller Root

Three API controllers (Admin, Customer, Operator) all needed to read the JWT claim *NameIdentifier* as an integer user-id. Before refactoring this was duplicated as *GetUserId()* in each file. Now it lives once in an abstract base controller.

```
// Controllers/AppControllerBase.cs

public abstract class AppControllerBase : ControllerBase
{
    protected int GetCurrentUserId()
    => int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!);
}

// Concrete controllers simply inherit:
```

```
public class AdminController : ApplicationControllerBase { ... }

public class CustomerController : ApplicationControllerBase { ... }

public class OperatorController : ApplicationControllerBase { ... }
```

- ✓ `GetCurrentUserId()` declared once — used in all three controllers via inheritance.
- ✓ `'abstract' + ':' ControllerBase` combines framework contract with custom logic cleanly.

What Could Be Better

✗ `BaseEntity` does not define any abstract methods — it is technically more of a concrete base class. Adding an abstract `Validate()` method would force each model to implement domain-level validation.

✗ `Payment` model does NOT inherit `BaseEntity` (it redeclares `public int Id` manually). Inconsistent base class usage across the domain.

```
// CURRENT (inconsistent):

public class Payment

{

    public int Id { get; set; } // ← manually redeclared!

    ...

}

// BETTER:

public class Payment : BaseEntity { ... }
```

2. Partial Class

A **partial class** splits one class definition across multiple files. It is most useful when a class is very large or when generated code must coexist with hand-written code.

2.1 Usage in the Project

Partial classes were **not explicitly used** in this project. The EF Core toolchain generates migration files (in the Migrations/ folder) that internally use partial classes to separate the designer snapshot from the migration logic — but this is auto-generated, not hand-crafted.

2.2 Where It Could Have Helped

The two largest service files — **CustomerService.cs (26 KB)** and **OperatorService.cs (27 KB)** — contain 400–600 lines each. Partial classes would split these into logical sub-files without breaking the single-class guarantee that DI requires.

```
// PROPOSED SPLIT for CustomerService:

// CustomerService.Search.cs

public partial class CustomerService : ICustomerService
{
    public async Task< SearchBusesAsync(...) { ... }
    public async Task GetScheduleDetailsAsync(...) { ... }
}

// CustomerService.Booking.cs

public partial class CustomerService
{
    public async Task BlockSeatsAsync(...) { ... }
    public async Task CreateBookingAsync(...) { ... }
    public async Task CancelBookingAsync(...) { ... }
}

// CustomerService.Profile.cs

public partial class CustomerService
{
    public async Task GetProfileAsync(...) { ... }
    public async Task UpdateProfileAsync(...) { ... }
}
```

✓ DI registration stays the same — `services.AddScoped()`

✓ Each partial file has a clear, single responsibility — easier to navigate.

✗ Not applied in the current codebase — both large service files are monolithic single files.

3. Model Preparation

3.1 Domain Models (EF Core Entities)

The project correctly separates domain entities from API payload shapes. EF Core navigation properties use lazy-safe initializers and nullable annotations.

```
// Models/Booking.cs – rich domain model

public class Booking : BaseEntity
{
    public int CustomerId { get; set; }

    public int ScheduleId { get; set; }

    public decimal TotalAmount { get; set; }

    public decimal PlatformFee { get; set; }

    [Column("GST")]
    public decimal Gst { get; set; }

    public BookingStatus Status { get; set; } = BookingStatus.Confirmed;

    public DateTime BookedAt { get; set; } = DateTime.UtcNow;

    public decimal? RefundAmount { get; set; }

    // Navigation properties

    public User Customer { get; set; } = null!;

    public BusSchedule Schedule { get; set; } = null!;

    public ICollection Passengers { get; set; } = new List<>();

    public Payment? Payment { get; set; }

}
```

- ✓ **null! guards make intent explicit — EF will always populate navigation properties.**
- ✓ **Enum types (BookingStatus, BusStatus, UserRole) used instead of raw strings.**
- ✓ **Computed/domain fields (BookedAt) kept separate from inherited CreatedAt.**

3.2 Enums as Typed State

All status fields use strongly-typed enums, preventing magic strings.

```
public enum BookingStatus { Confirmed, Cancelled, Refunded }

public enum BusStatus { Pending, Active, Down, Removed }

public enum ApprovalStatus { Pending, Approved, Disabled }

public enum UserRole { Customer, Operator, Admin }
```

```
public enum PaymentStatus { Pending, Success, Failed }
```

- ✓ Type-safe state transitions — compiler rejects invalid status values.
- ✓ EF Core stores these as integers — efficient in the database.

3.3 DTOs — Data Transfer Objects

All 437 lines of DTO definitions are consolidated in a single file **DTOs/Dtos.cs**, grouped by domain section with clear comment banners. This is both a strength and a weakness.

[illegible]

- ✓ Email DTOs were correctly migrated from EmailService.cs to the DTOs layer.
- ✓ Grouping by concern with comment banners makes navigation easy.
- ✗ All 30+ DTOs in one file — any change requires opening a 437-line file.
- ✗ No base DTO class — RegisterDto and OperatorRegisterDto share Name/Email/Phone/Password but there is no shared parent class to enforce DRY.

3.4 Computed Properties in DTOs

```
// BookingListDto – computed property (no setter, derived from stored values)

public class BookingListDto
{
    public decimal TotalAmount { get; set; }

    public decimal PlatformFee { get; set; }

    public decimal Gst { get; set; }

    public decimal GrandTotal => TotalAmount + PlatformFee + Gst; // computed!
}
```

✓ GrandTotal is computed on the fly — no risk of stale cached value in the DTO.

4. Interfaces & Abstraction

Every service in the project has a corresponding interface. Controllers depend only on the interface — never on the concrete class directly.

```
// IAdminService.cs – pure contract, no implementation

public interface IAdminService

{

    Task< > GetOperatorsAsync(string? status);

    Task ApproveOperatorAsync(int adminId, int operatorId);

    Task DisableOperatorAsync(int adminId, int operatorId);

    Task< > GetBusesAsync(string? status);

    Task CreateRouteAsync(int adminId, CreateRouteDto dto);

    Task GetRevenueAsync();

    // ... 8 more methods

}

// AdminController uses the interface – not the class:

public class AdminController : ApplicationControllerBase

{

    private readonly IAdminService _svc;

    public AdminController(IAdminService svc) => _svc = svc; // DI

}
```

- ✓ **Dependency Injection (Program.cs) wires AdminService → IAdminService at runtime.**
- ✓ **AdminController can be unit-tested by passing a mock IAdminService.**
- ✓ **4 interfaces defined — IAdminService, ICustomerService, IOperatorService, IAuthService.**
- ✗ **IEmailService exists but EmailService is injected concretely in some places too.**

5. Inheritance & Encapsulation

5.1 Inheritance Hierarchy

The project applies a clean two-level inheritance tree:

Layer	Base	Derived Classes
Models	BaseEntity (abstract)	User, Bus, Booking, BusOperator, BusSchedule, BusLayout, BusRoute, SeatBlock
Controllers	AppControllerBase (abstract)	AdminController, CustomerController, OperatorController

Framework	ControllerBase (ASP.NET)	AppControllerBase → all controllers
-----------	--------------------------	-------------------------------------

5.2 Encapsulation

All model properties use C# auto-property syntax with sensible defaults. The service layer encapsulates all business logic — controllers never touch DbContext or raw SQL.

```
// Encapsulation in action – controller knows nothing about DB:

[HttpPut('operators/{id:int}/approve')]

public async Task ApproveOperator(int id)

{

    var result = await _svc.ApproveOperatorAsync(GetCurrentUserId(), id);

    return Ok(result);

}

// All EF Core queries live inside AdminService – hidden from controller.
```

- ✓ Service layer fully encapsulates data access — controllers are thin.
- ✓ JWT claim parsing is encapsulated in AppControllerBase.GetCurrentUserId().
- ✗ Some service methods are very long (50–80 lines) — internal logic could be further encapsulated into private helper methods.

6. Polymorphism

Polymorphism lets callers treat different types uniformly. In this project it appears at two levels:

6.1 Interface Polymorphism (Runtime)

```
// Program.cs – DI registration

builder.Services.AddScoped();

builder.Services.AddScoped();

builder.Services.AddScoped();

// The ASP.NET runtime resolves the correct concrete class at runtime.

// In a test, you can pass a MockAdminService – same controller code, different behaviour.
```

✓ Full runtime polymorphism via DI — swap implementation without touching the controller.

6.2 Compile-Time (Method Overloading / Expression Bodies)

```
// Expression-bodied members – concise but polymorphic in spirit:

protected int GetCurrentUserId()

=> int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));

// Constructor injection shorthand across all controllers:

public AdminController(IAdminService svc) => _svc = svc;

public CustomerController(ICustomerService svc) => _svc = svc;

public OperatorController(IOperatorService svc) => _svc = svc;
```

✗ No method overriding (virtual/override) used in the domain — all services implement their interface from scratch. A shared abstract BaseService with common CRUD could reduce boilerplate.

7. Overall Assessment Summary

OOP Concept	Applied?	Rating	Key Note
Abstract Class (BaseEntity)	✓ Yes	★★★★★	All 8 models inherit — perfect usage
Abstract Class (AppControllerBase)	✓ Yes	★★★★■	Removes JWT duplication across 3 controllers
Partial Class	✗ No	★★■■■■	Large service files need splitting
Inheritance (Models)	✓ Yes	★★★★■	Payment class missed — does not extend BaseEntity
Inheritance (Controllers)	✓ Yes	★★★★★	Clean 3-level chain: ControllerBase→AppControllerBase→Concrete
Interface / Abstraction	✓ Yes	★★★★■	4 interfaces, DI wired; minor gaps in EmailService

Encapsulation	✓ Yes	★★★★■	Thin controllers; service methods slightly long
Polymorphism	Partial	★★★██	Runtime via DI; no virtual/override in domain
Model Prep (Entities)	✓ Yes	★★★★★	Rich domain models with enums, nullability, nav props
Model Prep (DTOs)	Partial	★★★██	One big Dtos.cs file; no base DTO hierarchy

8. Recommendations for Next Iteration

1. Fix Payment model to inherit BaseEntity — removes the one inconsistency in the domain hierarchy.
2. Split CustomerService.cs and OperatorService.cs using partial classes — each partial file covers one domain concern (Search, Booking, Profile, Revenue).
3. Introduce a base RegisterDto with Name, Email, Phone, Password — RegisterDto and OperatorRegisterDto both extend it (inheritance in DTOs).
4. Add a generic BaseService abstract class with shared helpers like GetOrThrowAsync(int id) — avoids repeating null-check patterns in every service.
5. Split Dtos.cs into feature folders: DTOs/Auth/, DTOs/Booking/, DTOs/Operator/ etc.
6. Add abstract Validate() to BaseEntity and implement business rules per model (e.g., Booking.Validate() checks TotalAmount > 0).
7. Use virtual/override for polymorphic cancel logic — a base CancelAsync on a shared service removes the duplicated refund-calculation logic in Admin and Customer services.